

Phase-Aware Reinforcement Learning Based Page Replacement Policy

Extended Preliminary Report and Algorithm Design

Pranav Gupta, Shambhavi Sharma, Mohak Joshi
Shiv Nadar Institute of Eminence

March 30, 2026

1 Introduction

Modern computing workloads such as databases, machine learning pipelines, and web servers exhibit highly dynamic memory access patterns. These workloads frequently transition between distinct behavioral phases such as sequential streaming, looping, and random access. Traditional page replacement algorithms such as LRU and LFU rely on static heuristics and are unable to adapt to such transitions, leading to suboptimal cache utilization and increased miss rates.

This work proposes a **Phase-Aware Reinforcement Learning (PA-RL)** based page replacement policy that dynamically adapts to workload characteristics in real time. Unlike classical approaches, the proposed system learns an eviction policy through interaction with the environment while simultaneously detecting workload phases.

2 System Overview

The system consists of four major components:

- **Phase Detector:** Extracts workload characteristics from recent access history
- **State Encoder:** Converts cache and workload information into a numerical state
- **RL Agent:** Selects eviction actions using a neural network
- **Training Module:** Updates the model using experience replay

The entire system is trained online without prior trace knowledge.

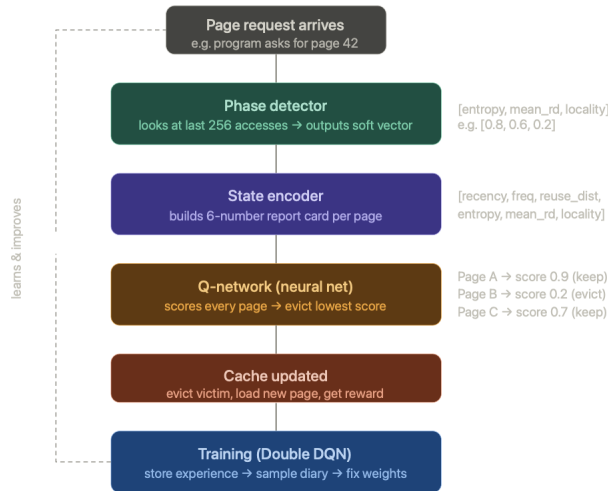


Figure 1: PA-RL system — how all 4 parts connect

3 Phase Detection

A sliding window of the last 256 memory accesses is maintained. From this window, three key statistics are computed:

3.1 Entropy

$$H = - \sum p_i \log_2 p_i$$

This measures the unpredictability of accesses. Low entropy indicates repetitive patterns, while high entropy indicates random behavior.

3.2 Mean Reuse Distance

The average number of accesses between repeated occurrences of the same page. Smaller values indicate strong temporal locality.

3.3 Locality

$$\text{locality} = \frac{\text{cache hits}}{256}$$

This measures the fraction of accesses that result in cache hits, reflecting how effectively the working set fits within the cache.

3.4 Phase Vector

$$\text{phase}_t = [H, \text{mean_rd}, \text{locality}]$$

This combines entropy, mean reuse distance, and locality into a compact representation of the current workload behavior.

3.5 Phase Change

$$\text{phase_change} = \frac{\|\text{phase}_t - \text{phase}_{t-1}\|_2}{\sqrt{3}}$$

This quantifies how rapidly the workload pattern is evolving by measuring the difference between consecutive phase vectors.

Algorithm 1 Phase Detector

- 1: Count frequency of pages in window W
 - 2: Compute entropy from frequency distribution
 - 3: Compute mean reuse distance using last-seen indices
 - 4: Compute locality as hit ratio
 - 5: $\text{phase} \leftarrow [\text{entropy}, \text{mean_rd}, \text{locality}]$
 - 6: $\text{phase_change} \leftarrow \|\text{phase} - \text{prev_phase}\|$
 - 7: Update prev_phase
 - 8: **return** $\text{phase}, \text{phase_change}$
-

4 State Representation

Each page in the cache is represented by an 8-dimensional feature vector:

$$s = [\underbrace{\text{recency_rank}}_{\text{LRU}}, \underbrace{\text{frequency}}_{\text{LFU}}, \underbrace{\text{reuse_dist}}_{\text{staleness}}, \underbrace{\text{age}}_{\text{aging}}, \underbrace{\text{entropy}}_{\text{phase}}, \underbrace{\text{mean_rd}}_{\text{phase}}, \underbrace{\text{locality}}_{\text{phase}}, \underbrace{\text{phase_change}}_{\text{transition}}]$$

4.1 Feature Interpretation

- Recency captures short-term importance
- Frequency captures long-term importance
- Reuse distance captures gap between reuse
- Age captures cache residency duration
- Phase features capture workload behavior
- Phase change captures transition dynamics

Algorithm 2 State Representation

- 1: Compute recency rank using LRU ordering
 - 2: Compute frequency from sliding window
 - 3: Compute reuse distance
 - 4: Compute age using insertion time
 - 5: Extract phase features
 - 6: Combine all into state vector s
 - 7: **return** s
-

5 Eviction Policy

For each page p :

$$Q(s_p; \theta) \rightarrow \text{value score}$$

Pages with lower scores are more likely to be evicted.

5.1 Softmax-Based Selection

$$P(p) = \frac{e^{-Q(s_p)/\tau}}{\sum_k e^{-Q(s_k)/\tau}}$$

- High τ : exploration phase
- Low τ : exploitation phase

This replaces enables smoother exploration.

Algorithm 3 Softmax-Based Selection

- 1: **for** each page p in cache **do**
 - 2: Compute s_p
 - 3: Compute $Q(s_p)$
 - 4: **end for**
 - 5: Convert scores to probabilities using softmax
 - 6: Sample victim based on probabilities
 - 7: **return** victim
-

6 Reward Function

The reward function is redesigned to reflect eviction quality:

- Hit: +1
- Reuse < 32: -2
- Reuse 32-128: -1
- Reuse > 128: -0.2
- Cold miss: -1

This allows the model to distinguish between good and bad evictions.

Algorithm 4 Reward Function

```
1: if cache hit then
2:   return +1
3: else if page was previously evicted then
4:    $gap \leftarrow current\_time - eviction\_time$ 
5:   if  $gap < 32$  then
6:     return -2
7:   else if  $gap < 128$  then
8:     return -1
9:   else
10:    return -0.2
11:  end if
12: else
13:   return -1
14: end if
```

7 Training Methodology

7.1 Q-Learning

$$y = r + \gamma Q_{\text{target}}(s')$$

$$L = (y - Q(s; \theta))^2$$

7.2 Prioritized Experience Replay

Priority:

$$p_i = |\delta_i| + \epsilon$$

Sampling:

$$P(i) = \frac{p_i^\alpha}{\sum p_k^\alpha}$$

Algorithm 5 Training with Prioritized Replay

```
1: Sample batch using priority distribution
2: for each  $(s, r, s')$  do
3:   Compute target  $y$ 
4:   Compute TD error  $\delta$ 
5: end for
6: Compute loss
7: Update network weights
8: Update priorities using  $|\delta|$ 
```

8 Algorithm (Simplified Pseudocode)

Algorithm 6 PA-RL Main Loop

```
1: for each request  $r_t$  do
2:   Update sliding window  $W$ 
3:   Compute phase and phase change
4:   if  $r_t$  in cache then
5:     reward  $\leftarrow +1$ 
6:   else
7:     reward  $\leftarrow \text{ComputeReward}()$ 
8:     victim  $\leftarrow \text{SoftmaxSelection}()$ 
9:     Evict victim and insert  $r_t$ 
10:  end if
11:  Store  $(s, r, s')$  in replay buffer
12:  Train network
13: end for
```

9 Preliminary Observations

The updated system introduces several key advantages:

- Better eviction decisions through reward shaping
- Faster adaptation via phase change detection
- More efficient exploration using softmax
- Improved learning speed via prioritized replay
- Richer state representation capturing both local and global behavior

10 Expected Results

The system is expected to achieve:

- 10–25% reduction in miss rate over LRU/LFU
- Improved performance under mixed-phase workloads
- Near-optimal behavior without future knowledge

11 Conclusion

This work presents an adaptive, learning-based page replacement policy that integrates workload phase awareness with reinforcement learning. The extended V2.0 design introduces multiple enhancements that significantly improve both learning efficiency and decision quality. The system bridges the gap between static heuristic policies and optimal offline algorithms, offering a practical and scalable solution for modern memory management systems.